Implementación de rn en python

M8: Fundamentos de deep learning

|**AE3**: Implementar una red neuronal artificial utilizando python para resolver un problema de regresión y/o clasificación.

alkemy

Introducción👩‍💻

El desarrollo de redes neuronales artificiales ha sido potenciado significativamente gracias a la disponibilidad de herramientas de programación accesibles y poderosas como Python. Este lenguaje, junto con librerías como Keras y PyTorch, ha permitido a estudiantes, investigadores y profesionales crear modelos de aprendizaje automático sin necesidad de una infraestructura compleja o conocimientos avanzados de matemáticas aplicadas.

Este manual tiene como objetivo guiarte paso a paso en la **implementación práctica de una red neuronal** orientada a resolver tanto problemas de regresión como de clasificación. A lo largo de este contenido teórico, aprenderás sobre los conceptos clave que forman parte del entrenamiento y la evaluación de redes neuronales, como tensores, funciones de activación, optimizadores, regularización, overfitting y más. La comprensión de estos elementos es fundamental para diseñar modelos eficientes, generalizables y efectivos en distintos tipos de datos.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Diseñar e implementar una red neuronal en Python desde cero utilizando Keras o PyTorch.
- Resolver un problema de regresión y uno de clasificación con redes neuronales.
- Aplicar funciones de activación, funciones de pérdida, optimizadores y ajustar hiperparámetros clave como el learning rate y las epochs.
- Evaluar el rendimiento del modelo e implementar técnicas de regularización como Dropout para reducir el sobreajuste.
- Realizar predicciones con modelos entrenados sobre datos reales o nuevos.

El entorno Python para redes neuronales

Python se ha consolidado como el lenguaje líder en inteligencia artificial debido a su sintaxis simple, gran comunidad, y la amplia variedad de librerías especializadas. Para trabajar con redes neuronales artificiales, las bibliotecas más utilizadas son:

- **Keras**: ofrece una interfaz de alto nivel que permite crear y entrenar redes neuronales con pocas líneas de código. Se integra directamente con TensorFlow.
- **PyTorch**: permite mayor control del modelo gracias a su diseño dinámico. Es ampliamente usado en investigación.

Ambos entornos permiten trabajar con GPU, realizar experimentos reproducibles, visualizar resultados y escalar modelos a producción.

Característica	**Keras**	**PyTorch**
Facilidad de uso	Muy alto	Alta
Curva de aprendizaje	Baja	Media
Popularidad en investigación	Media	Alta
Documentación	Muy completa	Completa
Integración con TensorFlow	Nativa	Parcial

La elección entre Keras y PyTorch dependerá de los objetivos del usuario: Keras es ideal para comenzar, mientras que PyTorch es recomendable para proyectos más complejos o experimentales.

¿Qué es un tensor?

Un **tensor** es la unidad básica de datos en los frameworks de Deep Learning. Se puede entender como una generalización de matrices:

- **Escalar** → tensor de 0 dimensiones (número).
- **Vector** → tensor de 1D.

- **Matriz** → tensor de 2D.
- **Tensor n-dimensional** → datos complejos como imágenes, secuencias o lotes de datos.

Por ejemplo, una imagen en color de 64x64 píxeles con 3 canales (RGB) se representa como un tensor de dimensiones (64, 64, 3). Durante el entrenamiento, los tensores fluyen a través de las capas de la red, sufriendo transformaciones lineales y no lineales hasta producir una predicción.

En Keras o PyTorch, los tensores se crean automáticamente al cargar datos y son manipulados mediante operaciones matemáticas optimizadas para CPU y GPU.

Diseño de una red neuronal para resolver un problema de regresión

Una red para regresión predice un valor numérico continuo. Su estructura típica es:

- Capa de entrada: igual al número de variables explicativas.
- Varias capas ocultas con activación **ReLU**.
- Capa de salida: una sola neurona sin activación o con **lineal**.
- Función de pérdida: **mean squared error (MSE)**.
- Métricas: **mean absolute error (MAE)**, **R^2**, etc.

Ejemplo conceptual en Keras:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

modelo = Sequential([
    Dense(64, activation='relu', input_shape=(n_features,)),
    Dense(32, activation='relu'),
    Dense(1)  # Salida regresión
])
modelo.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

Este modelo será entrenado sobre datos reales y evaluado en conjunto de validación. Ajustes como el número de capas y neuronas dependerán de la complejidad del problema.

Diseño de una red neuronal para resolver un problema de clasificación

En clasificación, el objetivo es asignar una etiqueta a una entrada. Dependiendo del número de clases, se debe ajustar la capa de salida:

- **Binaria:** una neurona con activación **sigmoid**.
- **Multiclase:** n neuronas con **softmax**.

La función de pérdida más usada es **categorical_crossentropy** o **binary_crossentropy**.

Ejemplo conceptual en PyTorch:

```
import torch.nn as nn
class RedClasificacion(nn.Module):
    def __init__(self):
        super(RedClasificacion, self).__init__()
        self.layer1 = nn.Linear(n_features, 64)
        self.layer2 = nn.Linear(64, 32)
        self.output = nn.Linear(32, n_classes)

    def forward(self, x):
        x = torch.relu(self.layer1(x))
        x = torch.relu(self.layer2(x))
        return torch.softmax(self.output(x), dim=1)
```

Este modelo es adecuado para tareas como clasificación de imágenes, textos o señales.

Funciones de activación

Las funciones de activación permiten introducir no linealidad en la red. Las más comunes son:

Función	Fórmula / Notación	Uso habitual
ReLU	max(0, x)	Capas ocultas, rápido y estable

Sigmoid	1 / (1 + e^-x)	Clasificación binaria (salida)
Softmax	e^xi / ∑e^xj	Clasificación multiclase
Tanh	(e^x - e^-x)/(e^x + e^-x)	Menos usada, reemplazada por ReLU

La elección depende de la tarea y la capa. ReLU es la más eficiente en capas intermedias por su simplicidad computacional.

Entrenamiento: función de pérdida, optimizadores, learning rate, epochs

Función de pérdida

Evalúa qué tan bien está funcionando la red:

- **Regresión**: mean_squared_error, mean_absolute_error
- **Clasificación**: binary_crossentropy, categorical_crossentropy

Optimizadores

Se encargan de ajustar los pesos de la red:

- **SGD** (descenso de gradiente estocástico)
- **Adam** (adaptativo, recomendado en la mayoría de los casos)

Learning rate

Define cuánto se ajustan los pesos por iteración. Si es demasiado alto, puede no converger; si es muy bajo, el entrenamiento será lento.

Epochs

Cada epoch representa una pasada completa por los datos de entrenamiento. Se debe ajustar según el comportamiento de la pérdida y la validación.

Evaluación y optimización de la red

La evaluación se realiza comparando el rendimiento en un conjunto de validación con el conjunto de entrenamiento. Las métricas más comunes son:

- **Accuracy**
- **MAE/MSE**
- **Precision / Recall / F1-score**

Se utilizan técnicas como:

- Validación cruzada.
- Early Stopping (detener el entrenamiento si no mejora).
- Ajuste de hiperparámetros (número de capas, neuronas, learning rate).

Validación y regularización: el problema del sobreajuste en las redes neuronales

El sobreajuste ocurre cuando la red memoriza los datos de entrenamiento y no generaliza a datos nuevos. Algunas técnicas para mitigarlo son:

- **Dropout**: desactiva aleatoriamente neuronas durante el entrenamiento.
- **L2 regularization**: penaliza pesos altos.
- **Aumento de datos**: en problemas de imagen o texto.
- **Reducir la complejidad del modelo**.

Dropout en Keras:

```
from tensorflow.keras.layers import Dropout
modelo.add(Dense(64, activation='relu'))
modelo.add(Dropout(0.5))
```

Esto mejora la generalización y reduce el riesgo de sobreentrenamiento.

Realizando predicciones con el modelo neuronal

Una vez entrenado y evaluado, el modelo puede ser utilizado para predecir sobre nuevos datos con el método .predict() (Keras) o el método .forward() seguido de .argmax() en PyTorch.

Ejemplo:

```
predicciones = modelo.predict(nuevos_datos)
```

Las predicciones pueden ser probabilísticas (probabilidad de pertenecer a una clase) o determinísticas (clase predicha).

Cierre ✍️

El desarrollo e implementación de redes neuronales en Python representa una puerta de entrada accesible y poderosa al mundo del aprendizaje profundo. Gracias a bibliotecas como Keras y PyTorch, hoy es posible construir modelos efectivos con pocas líneas de código y enfocarse en el análisis de datos y la lógica del problema en lugar de en la infraestructura técnica. Dominar estos entornos no solo mejora la productividad, sino que también permite experimentar rápidamente con distintas configuraciones, tareas y datos.

A lo largo de este manual se exploraron los principales elementos que intervienen en la construcción de una red neuronal: desde la comprensión de los tensores, el diseño de arquitecturas para clasificación y regresión, hasta el entrenamiento utilizando funciones de pérdida, optimizadores, y técnicas de regularización. También se abordaron herramientas de evaluación y predicción, claves para validar el desempeño del modelo y asegurar que pueda generalizar correctamente a nuevos datos.

Contar con estas habilidades te permitirá implementar soluciones concretas a problemas reales en diversos campos como la salud, finanzas, automatización o análisis de texto. Este es solo el comienzo: con la base adquirida, podrás continuar explorando arquitecturas más complejas, aplicar modelos preentrenados y optimizar redes para producción en entornos reales.

¡Nos vemos de nuevo, más adelante! 👋

Referencias📚

- TensorFlow. (2024). *Keras documentation*. TensorFlow. Recuperado de https://keras.io/
- PyTorch. (2024). *Learn the basics*. PyTorch. Recuperado de https://pytorch.org/tutorials/beginner/basics/intro.html
- IBM. (2023). *What is a tensor in machine learning?* IBM Cloud. Recuperado de https://www.ibm.com/cloud/blog/what-are-tensors

¡Muchas gracias!

Nos vemos en la próxima lección

alkemy